
Observability Debt in Agent Evaluation: Toward a Telemetry Layer for AgenticOS

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Agentic AI systems are evaluated today through three disjoint public infrastructures—
2 leaderboards, trace corpora, and telemetry logs—none of the public sources we exam-
3 ine jointly exposes the fields a resource-aware operating layer would need: task identity,
4 model identity, scaffold and policy-configuration identity, outcome, token/cache usage, re-
5 tries, latency, and billing. We measure its consequences across three studies on public
6 data. First, using the Holistic Agent Leaderboard (HAL), raw dollar-cost rankings trans-
7 fer across some benchmark pairs (Spearman $\rho=0.74\text{--}0.93$) but are mostly absorbed by a
8 workload-persistent displayed-label cost propensity (pooled model $R^2=82.0\%$); success-
9 adjusted cost is markedly less portable (mean $\rho=0.59$ vs. 0.84). Second, using 977 exact
10 task–model–matched trajectory pairs from Open-SWE-Traces, scaffold choice changes ob-
11 servable execution burden at fixed task and model (OpenHands/SWE-agent turn ratio 0.85 ,
12 $[0.82, 0.87]$), and the widely assumed “expensive failure” premium is not detected under
13 exact task matching (80%-power threshold: premium fractions ≥ 0.65 ; observed 0.44--
14 0.49). Third, auditing two independent telemetry corpora (8,058 and 1,017 sessions), we
15 characterize the resource anatomy—93.7% mean cache-read share, $4.29\times$ conditional tool-
16 error cascades replicated across sources, top-1%-of-sessions holding 48.3% of input-token
17 volume—that a scheduler or router would need but that no benchmark currently attaches
18 to an outcome. These studies are complementary rather than linked: Studies 1–2 show
19 that transferred raw-cost rankings and unconditioned failure premiums confound *which*
20 *workloads were attempted* with *how agents behaved on them*, while Study 3 shows that
21 the execution variables hidden beneath those aggregates are themselves heterogeneous,
22 heavy-tailed, and state-dependent. From this evidence we derive a joint telemetry schema
23 spanning identity, execution, resources, control, and outcome, and argue it belongs at the
24 operating-system layer: controlling execution and allocating resources requires observing
25 what those allocations depend on.

26 1 Why This Belongs at the OS Layer

27 Agentic AI systems persist state, invoke tools, retry failed actions, and accumulate context across long horizons.
28 Reasoning about their resource behavior—for scheduling, routing, cost budgeting, or governance—requires the
29 same unified telemetry that operating systems have long provided for conventional workloads: a common
30 schema linking *what ran*, *on what*, *under which policy*, *with what outcome*, and *at what resource cost*. We
31 use *observability debt* to name the accumulated inability of public agent-evaluation infrastructure to jointly
32 attribute resource consumption to task, model, execution policy, outcome, and system-level events. In every
33 public source we examined, this debt prevents reliable attribution of observed resource differences to workload,
34 model, scaffold, configuration, or execution behavior.

35 What makes this an *operating-system* responsibility rather than better benchmark bookkeeping? An agentic
36 OS occupies a specific functional position. It observes *identity* (which task, model, scaffold, and policy con-
37 figuration is running), *execution* (which tools act, what context accumulates), *resources* (tokens, cache, time,
38 memory, money), *control* (retry, terminate, route, budget), and *outcome* (did the task succeed). The same
39 resource-accounting principle motivates an OS-level treatment: the layer that controls execution and allocates
40 resources needs access to the telemetry required to make those decisions. Without per-round token components,
41 a memory manager cannot budget context growth; without configuration identity, a router cannot learn *why* one

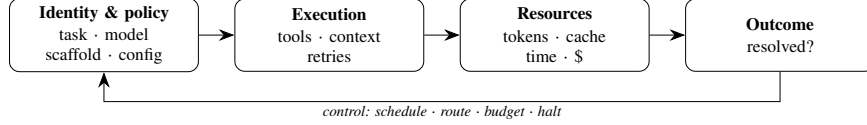


Figure 1: **The agentic OS control loop.** Each stage consumes exactly the schema fields derived in Section 6; the feedback arrow hosts scheduling, routing, budgeting, and halting decisions that today run on aggregate dollars because no examined public source closes this loop (Table 2).

label is cheap; without error-status streams, a governor cannot halt a spiraling session. Today, each of these decisions is made implicitly from aggregate dollar figures (Figure 1).

We treat three public data sources as stand-ins for what this observability layer would unify: a cost-and-outcome leaderboard (HAL), a task/model/scaffold-matched trajectory corpus (Open-SWE-Traces), and raw session-level usage telemetry (two independent corpora). Each exposes a different partial view. HAL links dollar cost to outcome but encrypts execution traces. Open-SWE-Traces exposes exact task/model/scaffold matching and resolved status but no tokens or billing. The telemetry corpora expose token, cache, and tool-call dynamics but no outcome or billing. **No public source we examined jointly exposes task, model, scaffold and policy configuration, outcome, token/cache usage, retries, latency, and billing**, the joint schema a resource-aware OS layer needs (Table 2). Importantly, combining these sources does not create record-level observability: their different execution units, identifiers, and release boundaries prevent resource use, outcome, and billing from being joined into a single per-execution record.

Our three studies are therefore complementary rather than linked—each probes a different dimension of the gap, and none can be joined to another at the record level. (1) A coverage-aware cost-portability study (HAL) shows that aggregate dollar cost conflates displayed-label propensity with benchmark difficulty and decorrelates under success adjustment. (2) A matched-trajectory study shows that execution structure differs measurably across scaffolds, while a popular failure-cost heuristic turns out to measure workload composition. (3) A cross-source telemetry audit shows that the execution variables hidden beneath aggregate cost are themselves highly heterogeneous, heavy-tailed, and state-dependent. Together they triangulate one gap; no pair of them jointly identifies a causal mechanism. This paper is an empirical requirements study for an AgenticOS telemetry layer, rather than an implementation of the layer itself.

2 Related Work

Kapoor et al. (2026) introduce HAL as cost-aware evaluation infrastructure spanning coding, web, science, and customer-service tasks (21,730 rollouts, roughly \$40,000 in evaluation cost). Kapoor et al. (2024) motivate joint accuracy–cost reporting and convex-hull frontier analysis, which we adopt as one of three decision-frontier definitions. Ndzomga (2026) asks whether task subsets preserve agent rankings within a single benchmark; we instead ask whether an *observed cost signal* predicts a *different* workload. Benchmark² studies cross-benchmark rank consistency for static LLM benchmarks; our setting adds interactive scaffolds, dollar cost, success adjustment, and a held-out workload test. SWE-Effi reports that failed attempts consume substantially more resources than successful ones; our matched design tests that claim at fixed task and does not detect the premium once task-difficulty confounding is removed (Section 4). Open-SWE-Traces supplies the exact matching that makes this test possible but lacks token/billing fields, motivating Section 6. Adjacent work sits on either side of this seam: commercial LLM tracing platforms expose per-deployment span usage but are not cross-benchmark evaluation infrastructure, while public evaluation releases strip execution detail; serving-side schedulers and KV-cache managers presuppose exactly the per-request visibility our audited sources lack. We quantify this seam on public data rather than propose another tracer or scheduler.

3 Study 1: Does a Cost Signal Travel Across Workloads?

3.1 Data and cohort

We use the public `all_leaderboards_costs_HAL.csv` snapshot distributed with Ndzomga (2026): 242 displayed evaluations across nine benchmarks, 11 scaffolds, and 29 displayed model labels. The CSV supplies accuracy and aggregate dollar cost but no token components, provider aliases, tool-call counts, retries, or per-task trajectories—the exact gap this paper is about.

The only scaffold repeated broadly enough is the displayed HAL Generalist Agent. Fixing it and matching exact label strings yields six workloads with adequate overlap—CORE-Bench Hard, GAIA, SciCode, ScienceAgent-Bench, SWE-bench Verified Mini, TAU-bench Airline—for a cohort of 86 rows and 25 labels (USACO has one Generalist observation and is excluded).

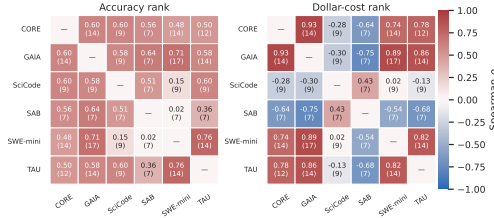


Figure 2: **Raw cost transfers; success-adjusted cost does not.** Pairwise Spearman rank correlations across benchmark pairs sharing ≥ 12 displayed labels. Raw dollar cost is strongly portable (mean 0.84); cost per expected success is not (mean 0.59), and only three CPS pairs survive Holm-corrected permutation testing.

3.2 Raw cost transfers; the mechanism behind it does not

Across six pairs with ≥ 12 shared labels, raw dollar-cost Spearman ρ ranges 0.74–0.93 (mean 0.84), surviving Holm-corrected permutation testing (Figure 2). A pooled label-plus-benchmark log-cost fixed-effects model explains $R^2=82.0\%$ (adjusted 72.7%); once label and benchmark propensity are removed, **no high-overlap residual-cost association survives** a 20,000-draw permutation test.

Leave-one-benchmark-out (LOBO) prediction succeeds for CORE ($\rho=0.78$, $N=14$, $p=0.0016$), GAIA (0.87, $N=16$), SWE-mini (0.90, $N=16$), TAU (0.83, $N=14$), all robust to every leave-one-label-out omission (e.g., CORE [0.73, 0.91], SWE-mini [0.88, 0.92]). It fails for SciCode (unstable, $[-0.50, 0.14]$; 2.1% mean accuracy, one-third zero-accuracy labels) and ScienceAgentBench (directionally negative, $[-0.77, -0.43]$; narrowest log-cost dispersion, $N=7$).

Interpretation: the portable component behaves as a workload-persistent displayed-label propensity—plausibly reflecting provider pricing, reasoning-mode defaults, or routing policy—that a router could in principle learn and reuse. but public data cannot identify *which*, since the distinguishing fields are not exposed.

3.3 Success-adjusted cost is a different, less portable ranking

Cost per expected success, $CPS_{i,b}(\epsilon) = C_{i,b} / \max(A_{i,b}, \epsilon)$ with primary floor $\epsilon=0.01$ (sensitivities at 0.001, 0.05, and zero-exclusion in the supplement), has mean $\rho=0.59$. Only three pairs survive correction: CORE–GAIA (0.87, [0.63, 0.96]), CORE–TAU (0.83, [0.31, 1.00]), GAIA–TAU (0.78, [0.42, 0.94]). The clearest raw-vs-CPS gap is GAIA–SWE-mini (0.60, [0.14, 1.13]): a label cheap in dollars is not reliably cheap per expected success elsewhere. No label tested on ≥ 5 workloads sits on every accuracy–cost frontier under discrete nondominance, convex hull, or 5% tolerance (o4-mini Low reaches 5/6 under tolerance; no universal winner under any strict definition).

Implication: Routing on displayed dollar cost therefore optimizes a signal whose portability degrades substantially once success is taken into account.

4 Study 2: What Matched Trajectories Reveal That Aggregates Cannot

HAL’s own trace archives are encrypted, so Study 1 cannot say *why* costs differ. We use an external unencrypted corpus—977 exact task–model pairs from OpenSWE-Traces for MiniMax-M2.5 under two scaffolds (OpenHands, SWE-agent), a reproducible but nonrandom four-shard boundary sample. The schema exposes resolved status, roles, tool calls, and text but not billed tokens or dollars; we treat turns, tool calls, and characters as trace-burden proxies.

4.1 Scaffold choice changes observable burden at fixed task

OpenHands has lower matched medians: 123 vs. 149 trajectory turns (ratio 0.85, [0.82, 0.87]), 61 vs. 74 tool calls (0.85, [0.82, 0.86]), 176,708 vs. 194,263 characters (0.95, [0.92, 0.98]). Resolved status shows no detected difference ($p=0.197$): scaffolds differ in execution structure without a detected accuracy trade-off here.

4.2 The “expensive failure” premium is not detected under exact task matching

Comparing failed vs. resolved trajectories *across* tasks reproduces SWE-Eff’s pattern (19–36% heavier failures). But among the **107 discordant pairs**—identical task, identical model, one scaffold resolved and the other failed—the failed side is heavier only 47–52 times out of 107 on every metric (two-sided binomial $p \geq 0.25$; ob-

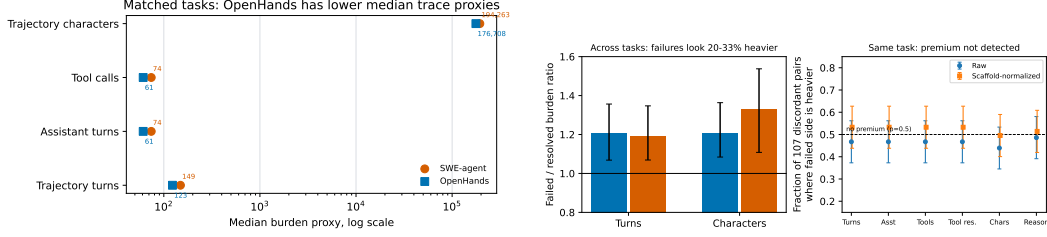


Figure 3: **(a)** Scaffold burden on 977 exact task-model pairs: OpenHands uses $\sim 15\%$ fewer median turns/tool calls than SWE-agent at fixed task and model. **(b)** The failure premium is not detected under exact task matching (80% power against premiums $\geq 15\text{pp}$). Left: naive cross-task ratios reproduce folklore (19–36% heavier failures). Right: among 107 discordant pairs, the failed side is heavier only 47–52/107 on every metric.

Table 1: **Telemetry anatomy across two independent corpora.** Cache metrics are computed within each corpus; a share and a total-volume ratio are different quantities and not interconvertible. Conditional-error ratios are computed on unrounded per-call rates.

	TraceLab	MIMO Claude Code
Sessions / LLM rounds	8,058 / 665,453	1,017 / 4,690
Median input / output tokens per round	132,092 / 249	n/a (constant-tuple audit)
Cache-read vs. uncached input	93.7% mean share ($\approx 14.9\times$)	753.0M / 46.3M tokens = $16.3\times$
Median final-vs-initial context	$3.6\times$ (P90 $9.9\times$)	n/a
Top-1% / top-10% input-token share	48.3% / 88.2%	— / 37.9%
Error cascade $P(\text{err} \text{err})/P(\text{err} \text{succ})$	$21.0/4.9\% = 4.29\times$	4.3% vs. 1.0% ($4.33\times$)
Errored calls retried same-tool / retry succeeds	88.3% / 77.9%	—
Consecutive-error run length (median/P90/max)	1 / 3 / 368	—
Tool-call latency, success vs. error (median)	140 ms / 313 ms	—
Tool-result size, success vs. error (median chars)	602 / 208	—

served fraction CIs [0.37, 0.57] raw, [0.43, 0.63] normalized). A pre-specified power calculation gives 80% power against a premium fraction ≥ 0.65 (failed heavier on 70+ of 107); the observed 0.44–0.49 does not resolve smaller effects ($< 15\text{pp}$). One model, nonrandom sample: this tests the heuristic here, not its general absence.

Interpretation: hard tasks make agents both burn more *and* fail more. An unconditioned failure multiplier measures *which tasks were attempted*, not a property of failing runs. Task-conditioned accounting requires exactly the joint schema no single public source provides.

5 Study 3: The Telemetry an OS Layer Would Need to See Directly

Study 1 shows that aggregate cost cannot explain *why* an agent is expensive; Study 2 shows that execution structure varies across agent systems at fixed task and model. This section shows that the execution variables hidden beneath aggregate cost are themselves highly heterogeneous, heavy-tailed, and state-dependent. Setting aside outcome and billing entirely: what does raw execution telemetry look like? We audit two independent coding-agent corpora (Table 1). **TraceLab:** 8,058 real-world sessions, 665,453 LLM rounds, 743,819 tool calls. **MIMO Claude Code:** 1,017 sessions, 4,690 rounds, single model. Neither links usage to outcome or billing; all summaries are compositional.

Context is the dominant resource, not generation. The median TraceLab round carries 132,092 input tokens against 249 output tokens (93.7% mean cache-read share); median final-round context is $3.6\times$ initial (P90 $9.9\times$). A leaderboard dollar total cannot distinguish price-driven from cache-policy- or accumulation-driven spend. Tool verbosity drives this: median next-input growth is 443 / 1,412 / 5,420 tokens below 1k / 1k–10k / above 10k tool-result characters.

Resource burden is heavy-tailed. The top 1% of sessions hold 48.3% of summed input volume; top 10% hold 88.2%. Median session input rises from 0.34M tokens at zero observed tool errors to 14.65M at six or more; mean-based budgeting would miss this tail-driven concentration (Figure 4).

Error cascades replicate across independent sources. After a successful tool call, the next call errs 4.9%; after an error, 21.0% ($4.29\times$). MIMO—different model, different site—replicates at $4.33\times$ (4.3% vs. 1.0%). Same-tool retry is common (88.3% of errored calls get one; 77.9% succeed), but the tail is extreme: consecutive-error runs have median length 1, P90 3, max 368. Failures are also slower yet less informative:

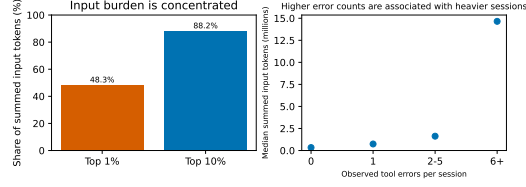


Figure 4: **Heavy-tailed burden.** Top-1%/top-10% concentration of summed input tokens (left axis) and median session input by observed-error bin (right): 0.34M at zero errors rising to 14.65M at 6+.

Table 2: **The observability gap.** Field coverage per public source we examined: ■ exposed, □ partial (display-label-only, degraded fidelity, or inferable), – absent. No examined source covers the joint schema; the union still leaves latency unlinked and round-level token fidelity unverifiable outside one corpus.

Field	HAL	Open-SWE	TraceLab	MIMO
Task identity	□	■	–	–
Model identity	□	■	□	■
Scaffold identity/version	■	■	□	■
Configuration / policy identity	□	–	–	–
Outcome / resolved status	■	■	–	–
Tokens / cache components	–	–	■	□
Retries / error status	–	□	■	□
Latency	–	–	■	–
Billing	■	–	–	–

median latency 313 ms vs. 140 ms (P95 85.6 s vs. 15.8 s) while returning 208 vs. 602 characters—more wall time, less signal.

Data-quality boundary. MIMO repeats one constant per-round usage tuple, so its within-session token dynamics are not validated; we use it only for session counts, totals, concentration, and content-derived error sequences.

6 An Evidence-Derived Joint Telemetry Schema

Table 2 shows the coverage gap; the missing rows are precisely those Sections 3–5 show to be load-bearing. Each field is tied to an analysis above or to the OS control function it unblocks; *evidence-derived* means we claim usefulness and necessity for those functions, not formal minimality. Keyed to the control function each feeds:

1. (*Attribution / exact matching*) **Task and model identity** (exact IDs, not display strings)—enables exact-matched designs; without it the Section 4 test is infeasible on public data.
2. (*Routing between scaffolds*) **Scaffold/harness identity and version**—attributes burden differences to scaffold design rather than model behavior; input to routing.
3. (*Configuration attribution*) **Configuration and policy identity** (reasoning mode, sampling params, budgets, routing/cache policy)—distinct from model identity; Study 1’s unexplained propensity component is plausibly the shadow of these missing fields.
4. (*Success-conditioned accounting*) **Outcome/resolved status**—conditions any cost statistic on success, closing the CPS portability gap.
5. (*Memory / budget management*) **Token components per round (input, output, cache-read, cache-creation)**—separates price from accumulation and cache policy; lets a memory layer budget context growth.
6. (*Governance / halting*) **Tool-call counts, retries, error status per call**—detects and prices $4.29\times$ cascade dynamics; a design motivation for governance policies that halt spiraling sessions—motivated here, not experimentally validated.
7. (*Real-time scheduling*) **Latency per call and per session**—present only as unlinked tool-call durations in one corpus; required for real-time scheduling.
8. (*Procurement / auditing*) **Billing, linked to all of the above**—the one field HAL exposes well, and exactly the field that is uninterpretable without the others.

Figure 1 places these fields at the stages of the control loop they close; Table 2 shows why, on public data alone, the loop does not close.

No public source we examined exposes this joint schema. We do not propose a storage format or API—a systems-design question outside our scope—but the measured consequences of its absence argue it belongs at

185 a shared operating-system layer rather than being re-derived piecemeal inside each benchmark, where today it
186 is not derived at all.

187 7 Limits

188 This is a secondary-data analysis with clear construct boundaries. Display labels do not fully identify prompts,
189 tools, budgets, or harness versions; the HAL cohort is sparse, unbalanced, mostly single-run. Propensity adjust-
190 ment is observational, not list-price control; CPS is an aggregate ratio, not the mean cost of successful runs.
191 Study 2 uses one model and a nonrandom four-shard sample whose proxies are not tokens or dollars, powered
192 only against strong premiums ($\geq 15\text{pp}$). Study 3 is observational and unlinked to outcomes or billing; MIMO’s
193 per-round fields are unverifiable and used only where supported, and its design implications (tail-aware bud-
194 geting, cascade-triggered halting) are motivations, not evaluated policies. None of this invalidates the schema
195 argument—each limit is itself an instance of the missing-field problem.

196 8 Conclusion

197 Across three complementary studies, we find that commonly used aggregate resource signals cannot be reliably
198 attributed to workload, execution, or outcome without telemetry fields that the public evaluation sources we
199 examined do not jointly expose. Raw-cost rankings transfer across workloads but are strongly associated with
200 a workload-persistent displayed-label cost propensity; success-adjusted cost reorders the ranking (surviving
201 correction for three of six pairs); the widely assumed failure-cost premium is not detected under exact task
202 matching. Beneath all three results sits one missing artifact: no examined public source jointly links task, model,
203 scaffold and configuration, outcome, token/cache usage, retries, latency, and billing. Because the layer that
204 controls execution and allocates resources is the natural owner of exactly this telemetry, we argue the schema
205 belongs at a shared operating-system layer for agentic AI—the role resource accounting plays in conventional
206 OSes—rather than being reconstructed independently, incompletely, and incompatibly inside each leaderboard
207 or trace release.

208 References

- 209 W. U. Ahmad, N. Ludwig, S. Majumdar, and B. Ginsburg. Open-
210 SWE-traces: Advancing dual-mode multilingual distillation for
211 software engineering agents. *arXiv preprint arXiv:2606.16033*,
212 2026.
213 Z. Fan, K. Vasilevski, D. Lin, et al. SWE-effi: Re-evaluating software
214 AI agent system effectiveness under resource constraints. *arXiv
215 preprint arXiv:2509.09853*, 2025.
- 216 S. Kapoor et al. AI agents that matter. *arXiv preprint
217 arXiv:2407.01502*, 2024.
218 S. Kapoor et al. Holistic agent leaderboard: The missing infrastruc-
219 ture for AI agent evaluation. *arXiv preprint arXiv:2510.11977*,
220 2026.
221 Franck Ndzomga. Efficient benchmarking of AI agents. *arXiv
222 preprint arXiv:2603.23749*, 2026.
223 Q. Qian, C. Huang, et al. Benchmark²: Systematic evaluation of
224 LLM benchmarks. *arXiv preprint arXiv:2601.03986*, 2026.

225 NeurIPS Paper Checklist

226 1. Claims

227 Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions
228 and scope?

229 Answer: [Yes]

230 Justification: All claims in the abstract and introduction are directly supported by the three public-data studies
231 (Sections 3–5) using pre-specified permutation/Holm tests, bootstrap and binomial confidence intervals, and ex-
232 plicit power analyses (Sections 3–4); limitations are stated in Section 7.

233 2. Limitations

234 Question: Does the paper discuss the limitations of the work performed by the authors?

235 Answer: [Yes]

236 Justification: Section 7 (Limits) discusses the single-model matched sample, nonrandom trace sample, proxy (not
237 dollar) measures, observational telemetry, and the unverifiable MIMO per-round fields.

238 3. Theory assumptions and proofs

239 Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and
240 correct) proof?

241 Answer: [N/A]

242 Justification: This is an empirical paper; our decompositions are observational fixed-effects models whose as-
243 sumptions are stated (Section 3). There are no theoretical results requiring proof.

244 4. Experimental result reproducibility

245 Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of
 246 the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the
 247 code and data are provided or not)?

248 Answer: [Yes]

249 Justification: The analysis pipeline and frozen outputs will be released upon publication; all inputs are public
 250 (HAL CSV, Open-SWE-Traces, TraceLab, MIMO). Cohort construction, resampling, and permutation procedures
 251 are described in Sections 3–4.

252 **5. Open access to data and code**

253 Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully
 254 reproduce the main experimental results, as described in supplemental material?

255 Answer: [Yes]

256 Justification: All input datasets are public; analysis code and frozen outputs will be released upon acceptance
 257 (anonymized until then).

258 **6. Experimental setting/details**

259 Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were
 260 chosen, type of optimizer) necessary to understand the results?

261 Answer: [Yes]

262 Justification: Analytic settings are stated: CPS floor $\epsilon=0.01$ with sensitivities at 0.001, 0.05, and zero-exclusion;
 263 20,000-draw permutation tests; bootstrap seeds; minimum pairwise overlap $n \geq 12$ (Sections 3–4).

264 **7. Experiment statistical significance**

265 Question: Does the paper report error bars suitably and correctly defined or other appropriate information about
 266 the statistical significance of the experiments?

267 Answer: [Yes]

268 Justification: We report permutation confidence intervals, Holm-corrected permutation tests, bootstrap intervals,
 269 and binomial sign tests with a pre-specified 80% power calculation (Sections 3–4).

270 **8. Experiments compute resources**

271 Question: For each experiment, does the paper provide sufficient information on the computer resources (type of
 272 compute workers, memory, time of execution) needed to reproduce the experiments?

273 Answer: [Yes]

274 Justification: Analyses are lightweight computations over public CSV and trace files run on a standard workstation
 275 (Apple M-series, 32 GB RAM); total runtime is on the order of minutes, with no GPU or large-scale compute
 276 required.

277 **9. Code of ethics**

278 Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics
 279 <https://neurips.cc/public/EthicsGuidelines>?

280 Answer: [Yes]

281 Justification: This is a secondary analysis of public evaluation artifacts; no human subjects, no harmful content
 282 generation, and no privacy-sensitive data are involved.

283 **10. Broader impacts**

284 Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the
 285 work performed?

286 Answer: [Yes]

287 Justification: Section 7 notes that improved evaluation infrastructure is broadly positive (better resource awareness
 288 and cost/efficiency) and identifies no direct negative societal impact; each documented gap is a missing-field
 289 problem rather than an enabled harm.

290 **11. Safeguards**

291 Question: Does the paper describe safeguards that have been put in place for responsible release of data or models
 292 that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

293 Answer: [N/A]

294 Justification: No high-risk assets are released; we release analysis code and a proposed telemetry schema, not
 295 models or scraped datasets with misuse potential.

296 **12. Licenses for existing assets**

297 Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly
 298 credited and are the license and terms of use explicitly mentioned and properly respected?

299 Answer: [Yes]

300 Justification: HAL CSV, Open-SWE-Traces, TraceLab, and MIMO are public research releases and are cited in
301 the References.

302 **13. New assets**

303 Question: Are new assets introduced in the paper well documented and is the documentation provided alongside
304 the assets?

305 Answer: [\[Yes\]](#)

306 Justification: We release the analysis pipeline, figures, and the proposed telemetry schema (Sections 5–6); they
307 are documented in the accompanying repository.

308 **14. Crowdsourcing and research with human subjects**

309 Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text
310 of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

311 Answer: [\[N/A\]](#)

312 Justification: No crowdsourcing or research with human subjects is involved.

313 **15. Institutional review board (IRB) approvals or equivalent for research with human subjects**

314 Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed
315 to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based
316 on the requirements of your country or institution) were obtained?

317 Answer: [\[N/A\]](#)

318 Justification: No human subjects are involved.

319 **16. Declaration of LLM usage**

320 Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component
321 of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting
322 purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is
323 not required.

324 Answer: [\[Yes\]](#)

325 Justification: LLM-based assistants aided drafting and editing; all empirical claims and numbers were verified by
326 the authors against the underlying code and data. The LLM use did not impact the core methodology or originality
327 of the research.